KAVACH (कवच)

A Security-Governed Multi-Agent AI DevOps & Observability Platform

ONE-PAGE PROJECT CHARTER & SYNOPSIS · PHASE-1 SUBMISSION

Course Code & Name	CSE3101 — Agentic AI	**Credits / LDP**	3 (2-0-2)
Semester / Year	7th Semester, B.Tech CSE (AI & Data Science) · AY 2026–27	**Semester Duration**	27 Jul – 20 Nov 2026
Course Faculty	Dr. Soharab Hossain Shaikh (Coordinator) · Mr. Pranshu Tiwari	**Institution**	BML Munjal University, Gurugram
Prepared By	Dhruv Jain, Hemant Goyal, Rajat Yadav	**Repository**	github.com/jaindhruv1923/KAVACH

Executive Summary

Autonomous AI coding agents can now read a codebase, write a patch and open a pull request unsupervised — and that convenience is exactly what makes them dangerous inside a real enterprise repository. KAVACH is a deterministic governance layer that sits in front of any such agent: before a single token reaches an LLM, it screens the prompt for secrets and personal data, retrieves grounded repository context, and maps the blast radius of the change; after generation, it re-verifies every import against the real package registry and self-heals failing patches inside a sandbox. The platform is fully implemented and independently verifiable — 164 automated tests, a working FastAPI backend, and a live dashboard — not a slide-deck proposal.

1. Problem Definition

Rubric C1. Deploying unconstrained autonomous coding agents (in the mould of Devin, SWE-agent or Copilot Workspace) inside an enterprise codebase introduces four concrete failure modes that KAVACH is built to close:

- **Supply-chain slopsquatting —** agents hallucinate plausible-looking but non-existent package names (e.g. fastapi_jwt_vault_security); attackers pre-register those exact names on PyPI/npm with malicious payloads, so a routine pip install becomes a remote-code-execution vector.
- **Credential & PII leakage —** agents can copy hardcoded API keys, tokens or Indian national identifiers (Aadhaar, PAN) straight into a public commit or an external LLM prompt log.
- **Unbounded blast radius —** agents edit shared utility modules with no structural awareness of which downstream services depend on them, breaking things far outside the file they touched.
- **Prompt injection & non-convergent loops —** instructions hidden in a README, issue or commit message can hijack the agent's reasoning, while unconstrained debugging can spiral into unbounded latency and token spend.

None of these are hypothetical: they are the specific gaps that existing tools leave open, since standard SCA scanners (Dependabot, Snyk) only inspect static requirements.txt and never see a dynamically generated import statement, and a system-prompt instruction such as “please do not leak secrets” is advisory, not enforced.

2. Objectives and Measurable Outcomes

Rubric C2. Aligned to CO1 (multi-agent design), CO2 (privacy & security) and CO3 (interoperability & multi-modal tooling):

Objective	CO	Measurable Evidence / Target
Deterministic multi-agent orchestration across a 6-stage governed pipeline (Sentinel → Retriever → Blast-Radius Analyst → Coder → Package Firewall → Self-Healer)	**CO1**	10-stage finite-state machine live in app/agent/orchestrator.py; 164 / 164 automated tests passing
Privacy-preserving, air-gapped code generation for proprietary or sensitive repositories	**CO2**	Dual-engine LLM routing — Google Gemini 2.5 Flash (cloud) vs. local Ollama qwen2.5-coder:7b (air-gapped), zero network egress on the local path
Deterministic pre-execution security gate that never relies on the LLM to police itself	**CO1, CO2**	Shannon-entropy secret scanner (H > 4.5) + multilingual PAN/Aadhaar/phone/email/bank-detail detector; 1.00 precision, 1.00 recall, 1.00 F1 on the evaluation corpus
Open, standards-based interoperability with external developer tools	**CO3**	Model Context Protocol (JSON-RPC 2.0) server exposing security-scan, blast-radius and RAG-search tools to Cursor / Claude Desktop / VS Code
Change-impact awareness before a patch is ever applied	**CO1, CO3**	Python-ast bidirectional dependency graph; blast-radius scoring in < 12 ms on the demo repository

3. Proposed Methodology & System Architecture

Rubric C3. KAVACH implements a six-stage governed pipeline, backed by a 10-state finite-state-machine orchestrator:

Prompt → Sentinel Screening → Qdrant RAG Retrieval → AST Blast-Radius Analysis → LLM Synthesis → Package Firewall → Self-Healing Sandbox → Verified Patch

Five collaborating agent personas own these stages — SentinelAgent (pre-execution guard), RetrieverAgent (Qdrant semantic search), BlastRadiusAnalyst (AST dependency graph), DevOpsCoderAgent (evidence-grounded generation) and a SelfHealer that runs generated code in an isolated sandbox and repairs it against its own traceback, capped at three reflection cycles to prevent infinite loops.

Critical Play Areas — Choices and Rationale

Play Area	Choice Made	Why
LLM per agent	Zero-parameter deterministic engine for Sentinel & Blast-Radius Analyst; 22.7M-param sentence-transformer for retrieval; Gemini 2.5 Flash / Qwen2.5-Coder-7B for synthesis	Security and AST analysis need mathematical certainty, not probabilistic text — so they get zero-entropy, non-LLM engines. Only code synthesis, which needs reasoning, is routed to an LLM, and at low temperature to curb hallucinated APIs.

MAS architecture	Event-driven hybrid — Sentinel, Retriever and Blast-Radius Analyst run as concurrent async subscribers feeding a coordinator	Benchmarked against pure Sequential and Manager/Hierarchical patterns; the event-driven pattern isolates a failing stage instead of halting the whole run and cuts pre-generation latency versus a strictly sequential pipeline.
Communication protocol	Blackboard for internal agent state, Server-Sent Events for live telemetry, MCP (JSON-RPC 2.0) for external tool exposure	Peer-to-peer was rejected — it creates $O(N^2)$ message coupling as agents grow. Blackboard keeps internal state zero-overhead; MCP is the only protocol of the four that gives universal interoperability with outside IDEs.

4. Feasibility, Resource Planning & Tech Stack

Rubric C4. The stack was chosen so the whole platform runs on standard student hardware with no mandatory paid subscription — the only required key is a free-tier Google Gemini API key, and the local Ollama path removes even that dependency for sensitive workloads.

Layer	Technology
Backend & API	Python 3.11+, FastAPI (async REST + SSE streaming), Pydantic v2, SQLite audit trail
Retrieval / RAG	Qdrant vector store, sentence-transformers/all-MiniLM-L6-v2 (384-dim dense embeddings), code-aware AST chunking
Security engine	Regex + Shannon-entropy secret detector, multilingual PII detector, risk-adaptive policy engine, prompt-injection shield
Change-impact analysis	Python ast module — bidirectional import/call dependency graph, blast-radius scoring
LLM inference	Google Gemini 2.5 Flash (cloud) with an air-gapped local Ollama (qwen2.5-coder:7b) fallback for sensitive code
Interoperability	Model Context Protocol (MCP) JSON-RPC server; GitHub repository ingestion webhook
Frontend	Dark-mode Mission Control dashboard — HTML5 / vanilla JS / CSS3, Groq Whisper voice input
Testing & CI	pytest (164 tests: RAG, orchestration, generation, security, impact, API, integration), automated CI security gate

Current implementation status, independently reproducible via **kavach/verify_project.py**:

Metric	Measured	Target Benchmark
Automated test suite	**164 / 164 passing**	100% passing
Security-gate precision / recall / F1	**1.00 / 1.00 / 1.00**	≥ 0.95
RAG retrieval latency (top-3 cosine)	**< 45 ms**	< 150 ms
AST blast-radius analysis latency	**< 12 ms**	< 50 ms
REST endpoints live	**10 / 10**	100% up
Voice transcription latency (Groq Whisper)	**< 450 ms**	< 1000 ms

5. Individual Understanding & Responsibility

Rubric C5. The team is Dhruv Jain, Hemant Goyal and Rajat Yadav. The team contributed across the major phases of KAVACH, with Dhruv Jain serving as the Lead Architect & Developer, Hemant Goyal contributing to the AI pipeline, RAG and system integration, and Rajat Yadav contributing to demo coordination, presentation and QA. Each member can explain and defend their respective contribution during viva

Team Member	Role / Contribution Area
Dhruv Jain	Lead Architect & Developer — designed and implemented all five phases (RAG foundation, agent orchestration, evidence-grounded generation, the security engine, and change-impact analysis), the MCP server, and the dashboard; individually able to explain and modify any subsystem live during viva.
Hemant Goyal	Testing support, environment/dependency setup, and documentation review across phases.
Rajat Yadav	Demo coordination, presentation preparation, and QA assistance across phases.

Repository & Verification

Source: github.com/jaindhruv1923/KAVACH · Run: cd kavach/backend && uvicorn app.main:app --reload · Verify: python kavach/verify_project.py

_____	_____	_____
Dhruv Jain	Hemant Goyal	Rajat Yadav